

# TenantProof

Executed tenant-boundary evidence for Supabase SaaS

## Prove that Customer A cannot read or modify Customer B’s data.

Product strategy, page specifications, UX system, technical architecture, safety boundaries, commercial model, roadmap, release state, and AI continuation protocol.

|            |                                                                                                |
|------------|------------------------------------------------------------------------------------------------|
| SNAPSHOT   | 2026-08-31                                                                                     |
| STAGE      | GitHub Pages request-only preview live; commercial intake and payment closed                   |
| QUALITY    | 31/31 local gates passed; PR build and main deployment workflow succeeded                      |
| COMMERCIAL | \$349 verification / \$649 verification + repair; payments closed; no customers or revenue yet |

This report contains no customer data, credentials, real findings, testimonials, or revenue claims. Northstar CRM is fictional.

Snapshot: 2026-08-31 Product stage: GitHub Pages request-only technical preview live; static customer intake and payments closed Commercial stage: validated paid demand, no customer or revenue yet Next model objective: configure verified operator/commercial facts, then run a ten-buyer paid-pilot experiment through an authenticated channel

## Executive summary

TenantProof is a productized service for founders launching multi-tenant SaaS applications on Supabase/PostgreSQL. It executes a buyer-approved authorization matrix and produces redacted before/after evidence. The winning wedge is not “we enabled RLS”; it is “we proved the tenant and role boundaries that matter to your application.”

The product now exists on GitHub Pages as a polished request-only technical preview, interactive fictional report viewer, local scope worksheet, versioned report schema, demo-only runner, production hardening, operational templates, and a public Notion qualification path. PR #23 was squash-merged at 6bc99104bb91b32afb50c8c7d9b416d74566f8ed, and deployment run #40 succeeded.

## Why this product was selected

The project began with a broad goal: launch any product or service capable of earning at least \$100 within ten days. Multiple concepts were built or considered. The user rejected ideas that were easy to DIY, low urgency, aesthetically weak, or replaceable by generic AI/design labor.

Buyer-first research found recent paid requests for:

- a \$300 Supabase authorization review
- a \$500 Lovable/Supabase security audit
- a \$700 pre-launch Supabase/PostgreSQL review
- urgent repair of a real cross-tenant data leak
- recurring multi-tenant RLS implementation and audit work

TenantProof won because one legitimate engagement clears the revenue target, the risk is launch-critical, the deliverable is concrete, and generic AI cannot prove application-specific role semantics without executing the agreed controls.

## Product thesis

A scanner can detect missing RLS, broad policies, exposed functions, or public buckets. It cannot determine by itself whether a manager may see another team’s records, whether a coach may see an unassigned client, or whether a function preserves the same role semantics as the UI.

TenantProof turns those semantics into an explicit matrix:

actor × tenant × resource × operation × expected result

It then records the observed result, evidence, remediation, and retest status. The buyer receives an engineering decision artifact rather than a green badge.

## Implemented product

### Public site

- `index.html` — category framing, problem, evidence preview, method, pricing, boundaries, CTA
- `report.html` — interactive sample/local report viewer
- `methodology.html` — authorization and evidence method
- `request.html` — three-step local scope brief builder
- `privacy.html` — current pre-release privacy notice
- `terms.html` — current pre-release terms and limitations
- `404.html` — branded recovery route

## Product logic

- responsive navigation
- progressive reveal with reduced-motion fallback
- configuration-driven preview/ready state
- package selection through query parameters
- non-submitting local scope wizard
- copy and text-download brief actions
- local JSON report import with 2 MB limit
- report schema validation
- before/after phase comparison
- actor, area, operation, status, and text filters
- keyboard-selectable evidence rows
- printable report
- fictional demo runner locked against live mode

## Production preparation

- restrictive meta CSP on all HTML pages
- portable `_headers`
- canonical, Open Graph, and Twitter metadata
- sitemap, robots, manifest, `.nojekyll`, `llms.txt`
- 192/512 icons and 1200×630 social preview
- external runtime configuration
- release-readiness checker
- authorization, SOW, evidence, delivery, payment, intake, launch, marketing, and GitHub runbooks

## Public soft-launch surfaces

- public Notion qualification form with anonymous submission and no response visibility
- private Notion intake database with table and pipeline views

- public Notion launch page
- public fictional sample report
- synthetic end-to-end form submission completed and deleted

## Current quality result

Last completed browser QA:

- 22 required public files
- 7 HTML pages loaded
- 7 CSP-compatible pages
- 16 sample checks
- before: 11 pass, 4 fail, 1 unresolved
- after: 15 pass, 0 fail, 1 unresolved
- status filtering passed
- local report import passed
- local scope wizard passed
- mobile navigation passed
- no mobile overflow
- no console errors
- no external runtime requests
- no browser persistence or POST requests
- demo runner rejects non-demo mode

Do not treat this snapshot as permanent. Run `npm run quality` after every material change.

## Design direction

The visual system is intentionally premium, calm, and technical:

- deep ink and forest surfaces
- warm paper and off-white content areas
- restrained mint green as proof/positive accent
- red and amber reserved for evidence states
- large editorial headlines paired with compact evidence UI
- system fonts only
- generous spacing, strong hierarchy, minimal decoration
- no stock imagery, fake logos, or noisy security clichés

The design should feel like a trusted engineering dossier, not a hacker landing page or generic SaaS template.

## Commercial model

- Boundary Verification: \$349
- Verification + Repair: \$649
- larger/sensitive scope: manual quote
- request-only launch
- no unrestricted checkout
- recommended payment: operator-issued Stripe invoice after qualification and signed scope
- repair package can use two \$324.50 milestones
- raw evidence default: 14 days after accepted delivery

Payment stays closed until a real legal contracting party, jurisdiction, owned business contact, processor, tax/refund position, and delivery capacity are configured.

## Current blockers

- 1 Owned business contact, legal contracting identity, and jurisdiction are not configured.
- 2 The static site has no owned HTTPS intake/deletion route and remains in preview.
- 3 Static privacy/terms still contain pre-launch language.
- 4 No .well-known/security.txt until an owned security contact exists.
- 5 No processor, tax/refund position, or confirmed delivery capacity exists.
- 6 No authenticated approved acquisition channel is connected for individualized outreach.

## Immediate continuation plan

- 1 Treat current main and handoff/CURRENT\_STATE.json as the canonical source.
- 2 Keep static intake local-only and payment closed; do not invent launch facts.
- 3 Configure verified operator identity, jurisdiction, owned business contact, retention, and delivery capacity.
- 4 Add the owned HTTPS intake/deletion route, update legal copy, and pass the strict release gate before accepting customers.
- 5 Use an authenticated approved business channel for no more than ten highly matched buyer contacts.
- 6 Prefer one safely authorized paid pilot, then record objections, fulfillment time, margin, and outcome without publishing customer evidence.

## Mindset to preserve

- Buyer evidence before more features.
- Narrow and provable beats broad and impressive-sounding.
- Trust is created by limits, evidence, and operational discipline.

- A failed check is useful; an untested check is not a pass.
- One paid pilot matters more than traffic.
- Do not build a scanner dashboard, account system, or recurring SaaS until a buyer proves the need.
- Compete by making the purchase safer and the output more decision-useful, not by claiming all competitors are dead.

## Historical context

### RebuttalKit

A previous revenue product was technically built and deployed but commercially rejected. It was frozen with checkout closed. Do not reuse its live site as proof of TenantProof.

### Stay5

A habit/behavior product was completed locally but commercially rejected and never deployed. Frozen.

### App Store screenshot service / ClearCal

Rejected because buyers can use templates, AI, or inexpensive designers; the proposed price lacked durable value. No proposal was submitted.

### Relay migration and related shutdown services

Considered but rejected because reachable demand and differentiated fulfillment were weaker than TenantProof.

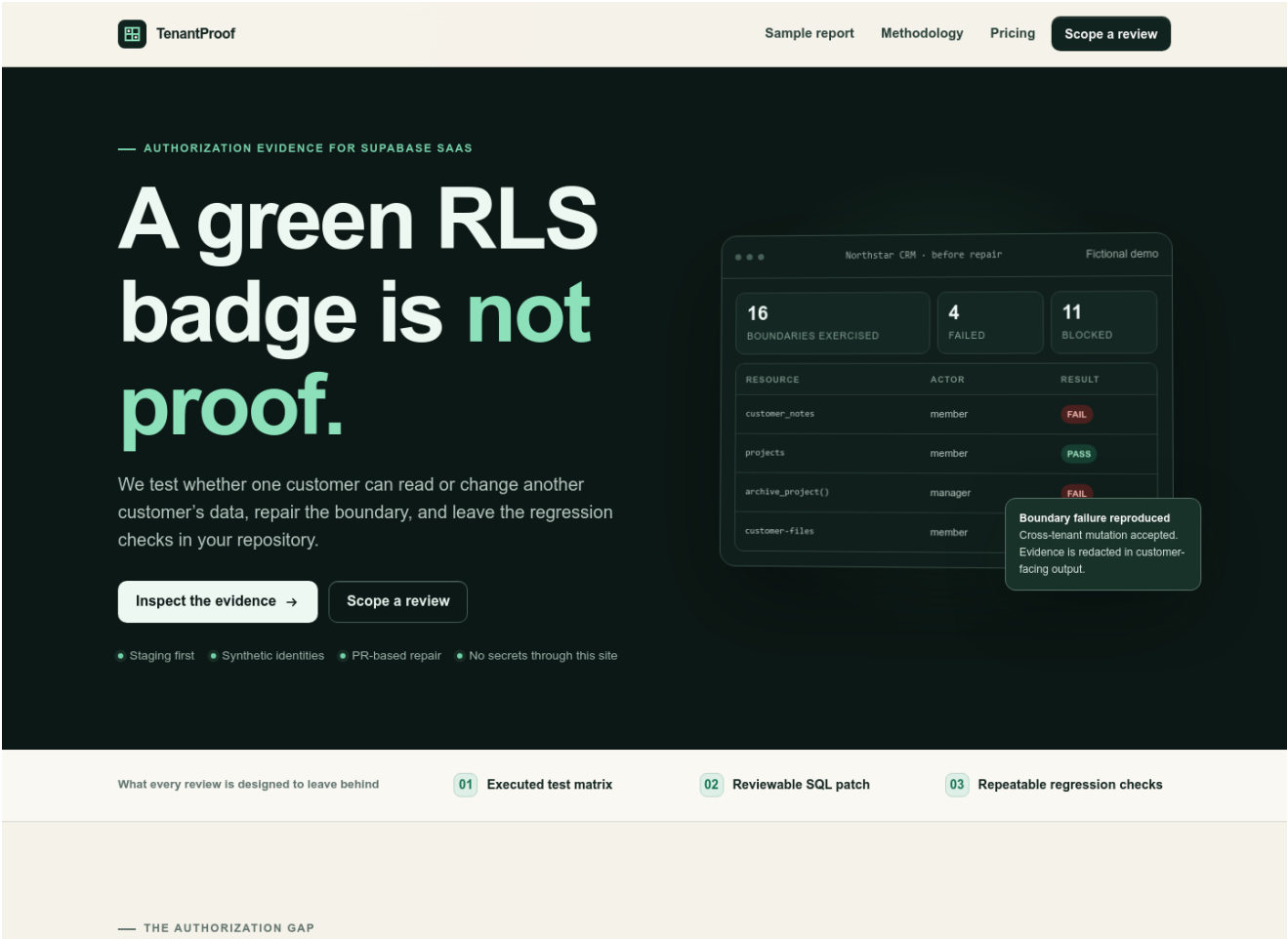
Full lessons are in REJECTED\_IDEAS\_AND\_LEARNINGS.md.

## Handoff integrity statement

This folder records strategy, implementation, decisions, limitations, and state. It does not contain customer secrets, production credentials, private data, or claims of work that did not happen. Any future agent must preserve that standard.

# Visual product reference

These images define the intended production quality. They are generated from the local fictional product and contain no customer evidence.



Desktop first viewport — category, evidence preview, and primary actions

TenantProof / Report viewer

— LOCAL-FIRST DELIVERABLE

# Every conclusion should point to an executed check.

Explore the fictional report, compare before and after, or open a compatible redacted JSON report entirely inside your browser.

## Open a TenantProof JSON report

Processed in this browser tab. The file is not uploaded or stored.

Choose report

Northstar CRM

TP-DEMO-0030 • Fictional staging demo

FICTIONAL DEMO

Desktop report viewer — local import, matrix controls, and evidence shell

TenantProof

TenantProof / Scope a review

PRIVATE LOCAL WORKSHEET

Describe the boundary. Keep the secrets.

This pre-launch form creates a review brief in your browser. It does not transmit the contents or authorize testing.

1 Package2 Application3 Private brief

STEP 1 OF 3

Choose the starting scope

Final scope is confirmed before payment or access.

Boundary Verification · \$349

Up to 12 tables, three roles, executed matrix, findings, and one retest.

Verification + Repair · \$649

Up to 25 tables, selected functions and storage, patches, regression checks, and one retest.

Review pricing

Describe application

NEVER PUT THESE IN THE BRIEF

Credentials belong in a separate, approved access process.

- Supabase service-role keys or database passwords
- Production customer records or database exports
- Personal information not necessary for scoping
- Credentials for GitHub, hosting, billing, or analytics
- A third party's target without written authorization

TenantProof

Evidence-driven tenant-boundary verification and repair for Supabase applications.

PRODUCT

Sample report

Methodology

Pricing

TRUST

Privacy

Terms

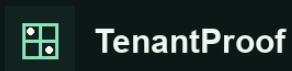
START

Scope a review

© 2026 TenantProof. Pre-launch preview.

This form does not transmit data.

Mobile scope worksheet — package, application surface, and non-sensitive brief



# A green RLS badge is not proof.

Executed tenant-boundary evidence for Supabase SaaS.

## BOUNDARY MATRIX

16

CHECKS

4

FAILED

11

BLOCKED

customer\_notes

FAIL

projects

PASS

archive\_project()

FAIL

customer-files

PASS

1200x630 social-preview asset

# Canonical source index

The PDF is a reviewable snapshot. The Markdown/JSON files below remain the editable source of truth.

- AGENTS.md
- handoff/HANDOFF\_REPORT.md
- handoff/CURRENT\_STATE.json
- handoff/PRODUCT\_SPEC.md
- handoff/PAGE\_AND\_SECTION\_SPECS.md
- handoff/UX\_DESIGN\_SYSTEM.md
- handoff/TECHNICAL\_ARCHITECTURE.md
- handoff/FEATURE\_AND\_FUNCTION\_INVENTORY.md
- handoff/REPORT\_ENGINE\_SPEC.md
- handoff/SECURITY\_PRIVACY\_AND\_SAFETY.md
- handoff/OPERATIONS\_AND\_INTAKE.md
- handoff/COMMERCIAL\_AND\_PAYMENTS.md
- handoff/GTM\_AND\_VALIDATION.md
- handoff/ROADMAP\_V1\_V2\_V3.md
- handoff/DECISION\_LOG.md
- handoff/REJECTED\_IDEAS\_AND\_LEARNINGS.md
- handoff/QUALITY\_GATES.md
- handoff/RELEASE\_AND\_REPOSITORY.md
- handoff/ASSET\_MANIFEST.md
- handoff/AI\_CONTINUATION.md

## Continuation rule

**A future agent reads AGENTS.md first, validates CURRENT\_STATE.json against the repository, runs the quality gate, and updates the handoff whenever behavior or decisions change.**